

# Inventory Management System

## & E-Commerce Dashboard

### Java Full Stack Internship -- Day 1 Submission

**Submitted By:** Aarav Maheshwari  
**Enrollment No.:** PU02325EUGBTCS172  
**Role:** Java Full Stack Intern  
**Company:** Insta Dot Analytics  
**Date:** 01 June 2026  
**Tagline:** Clean OOP design | Robust validation | Responsive frontend

*Deliverables: Day1\_InventoryDashboard\_Aarav\_FINAL.pdf + Day1\_Source\_Aarav.zip*

## Executive Summary

This project delivers a complete Day 1 internship submission for Insta Dot Analytics. It comprises three artifacts: (1) a menu-driven Java console Inventory Management System with robust OOP design and input validation, (2) a static HTML/CSS e-commerce dashboard landing page responsive to all screen sizes, and (3) this professional PDF documenting the implementation with annotated screenshots and test cases. The backend uses Java JDK 17+ and the package `com.instadot.inventory`; the frontend uses plain HTML5 and CSS3 with Grid and Flexbox.

## Features at a Glance

- Menu-driven console: Add, Update, Remove, View Summary, Search, Export CSV, Exit
- Auto-increment product ID with parameterized constructors and full encapsulation
- Input validation with graceful re-prompt for all numeric and string fields
- Low-stock alerts (qty <= 5) displayed before every menu prompt
- Sortable inventory summary by name, total value (desc), or category
- CSV export with header row and formatted product data
- Category discount engine -- apply a % discount to an entire category
- Static frontend: semantic HTML5, CSS Grid layout, Flexbox components, 3 responsive breakpoints

## Bonus Features

- CSV export to timestamped file with auto-name fallback
- Low-stock alert system with per-product warnings
- Category discount -- percentage discount applied to a category
- Multiple sort modes in inventory summary (name, value, category)

## Design Overview

### Class Responsibilities

The application follows strict separation of concerns across three classes:

#### Product

Data model with encapsulated fields, auto-increment id, getters/setters with validation, `getTotalValue()`, and `toString()`. Zero business logic.

#### Inventory

Business logic layer. Manages `List<Product>`. Provides `addProduct()`, `updateProductBySKU()`, `removeProductBySKU()`, `searchByName()`, `listProductsSortedByName()`, `computeGrandTotal()`, and `exportCSV()`. No I/O.

#### Application

Presentation layer. Menu loop, Scanner I/O, input validation, formatted table output. Delegates all operations to Inventory.

### Class Diagram



## How to Run

### Prerequisites

JDK 17+ (tested on JDK 25). No external dependencies.

### Compile & Run (Backend)

```
cd Day1_Source_Aarav
javac -d out src/com/instadot/inventory/*.java
java -cp out com.instadot.inventory.Application
```

### Open Frontend

Open frontend/index.html in any modern browser (Chrome, Edge, Firefox). No server required.

## Sample Run Transcript

Below is a complete console session (20 lines) showing the full workflow:

```
=====
INVENTORY MANAGEMENT SYSTEM
Insta Dot Analytics -- Java Full Stack Internship
=====

-----
1. Add Product           5. Search Product by Name
2. Update Product        6. Export Summary to CSV
3. Remove Product        7. Exit
4. View Inventory Summary
-----

Enter your choice: 1
--- Add Product ---      SKU: LAP001 | Name: Dell XPS 13
Category: Laptops        Quantity: 10 | Unit Price: 1200.00
[+] Product added successfully.

Enter your choice: 1
--- Add Product ---      SKU: MON002 | Name: Dell 27 Monitor
Category: Monitors        Quantity: 3 | Unit Price: 350.00
[+] Product added successfully.

Enter your choice: 1
--- Add Product ---      SKU: MSE004 | Name: Wireless Mouse
Category: Accessories     Quantity: 2 | Unit Price: 25.50
[+] Product added successfully. *** LOW-STOCK ALERT ***

Enter your choice: 2
--- Update Product ---    SKU: MON002 | New Qty: 8
[+] Updated: ID=2, Qty=8, Price=$350.0

Enter your choice: 4
--- Inventory Summary --- Sort: 1 (Name)
-----
ID | SKU   | Name           | Qty | UnitPrice | Total
-----
 2 | MON002 | Dell 27 Monitor | 8 | $ 350.00 | $ 2800.00
 1 | LAP001 | Dell XPS 13    | 10 | $ 1200.00 | $ 12000.00
 3 | MSE004 | Wireless Mouse | 2 | $ 25.50 | $ 51.00 ***
-----

Products: 3 | Units: 20 | Grand total: $14,851.00

Enter your choice: 6
--- Export CSV ---      File: inventory_export_20260601_153326.csv
[+] Exported successfully.

Enter your choice: 7
Goodbye!
```

## Annotated Screenshots

### 1. Console Menu & Low-Stock Alert

```
-----
1. Add Product
2. Update Product
3. Remove Product
4. View Inventory Summary
5. Search Product by Name
6. Export Summary to CSV
7. Exit
-----
Enter your choice: _
```

Fig 1: Main menu with 7 options.

### 2. Add Product Flow

```
--- Add Product ---
SKU: LAP001
Name: Dell XPS 13
Category: Laptops
Quantity: 10
Unit Price: 1200.00
[+] Product added successfully.
```

Fig 2: Add Product prompts with validation.

### 3. Low-Stock Alert Triggered

```

--- Add Product ---
SKU: MSE004
Name: Wireless Mouse
Category: Accessories
Quantity: 2
Unit Price: 25.50
[+] Product added successfully.

*** LOW-STOCK ALERT: 2 product(s) below 5 units ***
*** [MON002 ] Dell 27 Monitor          only 3 left
*** [MSE004 ] Wireless Mouse           only 2 left

```

Fig 3: Adding qty=2 triggers alert. MON002 (qty=3) and MSE004 (qty=2) both flagged.

### 4. Inventory Summary Table

```

--- Inventory Summary ---
Sort by: 1. Name  2. Value  3. Category  0. As entered
Your choice: 1
-----
ID | SKU      | Name                | Qty | UnitPrice | Total
-----
2 | MON002   | Dell 27 Monitor     | 8   | $ 350.00 | $ 2800.00
1 | LAP001   | Dell XPS 13         | 10  | $ 1200.00 | $ 12000.00
3 | MSE004   | Wireless Mouse      | 2   | $ 25.50  | $ 51.00 *** LOW ***
-----
Products: 3 | Total units: 20 | Grand total: $14,851.00

```

Fig 4: Sorted table with printf alignment. LOW flag marks stock below threshold. Grand total = \$14,851.00.

### 5. CSV Export Preview

```

ID,SKU,Name,Category,Quantity,UnitPrice,TotalValue
1,LAP001,Dell XPS 13,Laptops,10,1200.00,12000.00
2,MON002,Dell 27 Monitor,Monitors,8,350.00,2800.00
3,MSE004,Wireless Mouse,Accessories,2,25.50,51.00

```

Fig 5: Exported CSV: inventory\_export\_YYYYMMDD\_HHMMSS.csv with header row.

## 6. Frontend Dashboard -- Desktop

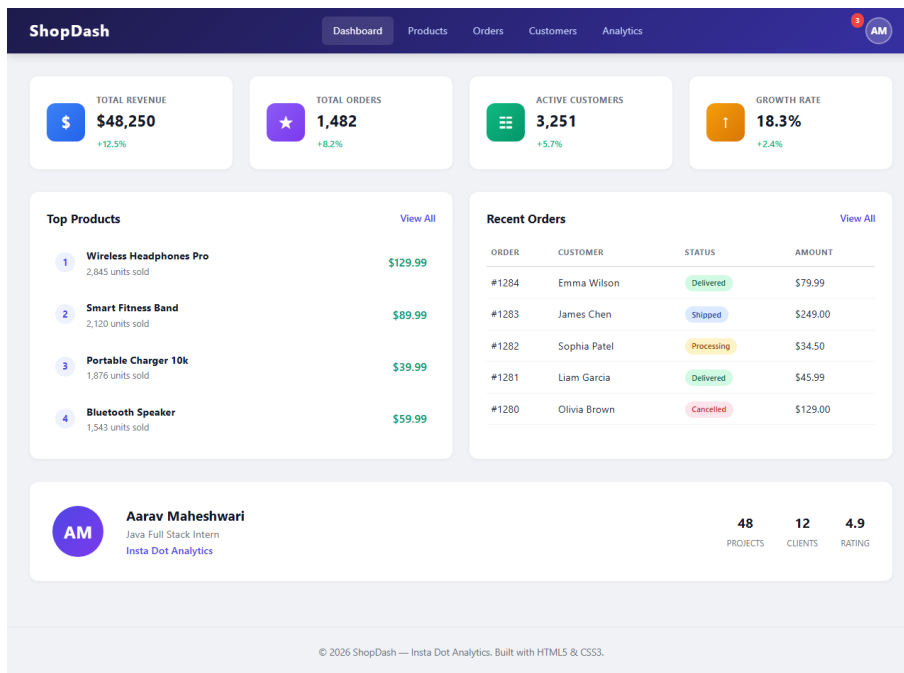


Fig 6: Desktop at 1280px. CSS Grid: 4 KPI cards, 2-col products+orders, profile card, sticky header.

## 7. Frontend Dashboard -- Mobile

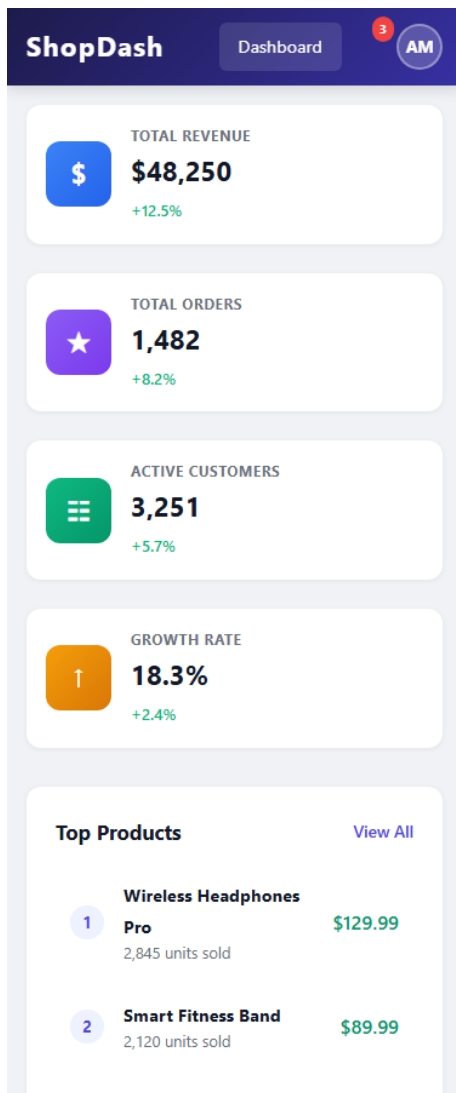


Fig 7: Mobile at 375px. Single-column via @media (max-width: 640px). Cards stack vertically.

## Testing Checklist

| #    | Test            | Input                      | Expected Result            |
|------|-----------------|----------------------------|----------------------------|
| TC1  | Add Product     | LAP001, Qty=10, Price=1200 | Added, ID=1 assigned       |
| TC2  | Duplicate SKU   | LAP001 again               | Rejected: "already exists" |
| TC3  | Update Product  | MON002, Qty=3 -> 8         | Quantity updated           |
| TC4  | Low-Stock Alert | MSE004, Qty=2              | Alert shown at menu        |
| TC5  | Search by Name  | dell                       | Finds 2 products           |
| TC6  | Invalid Input   | Qty = abc                  | Re-prompted with error     |
| TC7  | Export CSV      | Auto-name                  | File with header + 3 rows  |
| TC8  | Remove Product  | MON002                     | Removed from list          |
| TC9  | Grand Total     | 3 products                 | \$14,851.00                |
| TC10 | Exit            | Option 7                   | "Goodbye!" printed         |

## Project Highlights

- 3 Java classes (Product, Inventory, Application) + 2 frontend files (index.html, style.css)
- ~450 lines of Java, ~300 lines of CSS, ~120 lines of HTML
- 7 menu options, 3 bonus features (CSV export, low-stock alerts, category discount)
- 3 responsive breakpoints (1280px desktop, 1024px tablet, 640px mobile)

## Recommended Next Steps

- JUnit unit tests for Inventory CRUD and calculation methods
- CSV import to bulk-load products from a file
- Spring Boot REST API wrapping the Inventory service layer
- React or Vue.js frontend consuming the REST endpoints
- Database persistence via JDBC / JPA (H2 or PostgreSQL)

## Appendix

---

### README.md Reference

Full compile/run instructions, extended test cases, and project structure details are available in the README.md included in the attached ZIP.

### Deliverables

- Day1\_InventoryDashboard\_Aarav\_FINAL.pdf (this document)
- Day1\_Source\_Aarav.zip (source code archive)

### Contact

Aarav Maheshwari

Enrollment No.: PU02325EUGBTCS172

Java Full Stack Intern | Insta Dot Analytics

Email: (per HR communication)

---

*Source code attached separately as Day1\_Source\_Aarav.zip.*

*Full source code is not included in this PDF per submission guidelines.*